https://www.youtube.com/watch?v=b5vOnRmGqHw

Part 1: Introduction to TXT Files and Text Encoding

Text files (`.txt`) are commonly used to store logs, reports, scientific measurements, and other data.

Python can read TXT files using the built-in `open()` function or Pandas.

What is Text Encoding?

Text encoding determines how characters are stored as bytes.

Encoding	Description
ASCII	Basic English characters
UTF-8	Supports multilingual characters
ISO-8859-1	Older Western European encoding
Windows-1252	Common in older Windows files
UTF-16	Unicode encoding

Example: Café, Hyvää päivää, José

Using an incorrect encoding can produce unreadable characters or a `UnicodeDecodeError` .

UTF-8 is generally recommended for modern data processing.

In [1]:
```
# Create a UTF-8 file
with open("Dataset/students_utf8.txt", "w", encoding="utf-8") as f:
    f.write("Name: Ahmad\n")
    f.write("City: Kuopio\n")
    f.write("Text: Hyvää päivää\n")

# Create an ISO-8859-1 file
with open("Dataset/students_latin1.txt", "w", encoding="iso-8859-1") as f:
    f.write("Name: André\n")
    f.write("City: Montréal\n")
    f.write("Text: Café\n")

# Create a UTF-16 file
with open("Dataset/students_utf16.txt", "w", encoding="utf-16") as f:
    f.write("Name: Aisha\n")
```

```
    f.write("City: Helsinki\n")
    f.write("Text: Hello World\n")
```

Part 2: Reading TXT Files Using Python

Python provides the `open()` function for reading and writing text files.

Common file modes:

- `"r"` — Read a file
- `"w"` — Write a file
- `"a"` — Append data to a file

Two common reading methods:

- `read()` — Read the entire file
- `for line in file` — Read line by line

The `with` statement automatically closes the file.

In [3]:
```
with open("Dataset/students_utf8.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

```
Name: Ahmad
City: Kuopio
Text: Hyvää päivää
```

In [5]:
```
with open("Dataset/students_utf8.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line)
        #print(line.strip())
```

```
Name: Ahmad

City: Kuopio

Text: Hyvää päivää
```

In [8]:
```
# test this with 8 first
with open("Dataset/students_utf16.txt", "r", encoding="utf-16") as file:
    print(file.read())
```

```
Name: Aisha
City: Helsinki
Text: Hello World
```

Part 3: Encoding Errors and Their Solutions

A `UnicodeDecodeError` occurs when Python cannot decode file bytes using the selected encoding.

For example, a file saved using ISO-8859-1 may produce an error when read using UTF-8.

Common Solutions

Method	Description
Correct encoding	Reads the original characters correctly
`errors="ignore"`	Skips invalid characters
`errors="replace"`	Replaces invalid characters with `&#65533;`

Important: Ignoring or replacing characters may result in information loss.

In [10]:
```
# This code produces UnicodeDecodeError
with open("Dataset/students_latin1.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
Cell In[10], line 3
      1 # This code produces UnicodeDecodeError
      2 with open("Dataset/students_latin1.txt", "r", encoding="utf-8") as file:
----> 3     print(file.read())

File <frozen codecs>:325, in BufferedIncrementalDecoder.decode(self, input, fina
l)
    322 'Could not get source, probably due dynamically evaluated source code.'

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 10: invalid
continuation byte
```

In [12]:
```
with open("Dataset/students_latin1.txt", "r", encoding="iso-8859-1") as file:
    print(file.read())
```

```
Name: André
City: Montréal
Text: Café
```

In [14]:
```
with open("Dataset/students_latin1.txt", "r",
          encoding="utf-8", errors="ignore") as file:
    print(file.read())
```

```
Name: Andr
City: Montral
Text: Caf
```

In [16]:
```
with open("Dataset/students_latin1.txt", "r",
          encoding="utf-8", errors="replace") as file:
    print(file.read())
```

```
Name: Andr
City: Montr al
Text: Caf
```

Part 4: Detecting and Converting Text Encoding

Sometimes, the encoding of a text file is unknown.

The `charset-normalizer` library can estimate file encoding.

We can also convert older text files into UTF-8 for better compatibility.

Note: Automatic encoding detection is not always accurate. Always verify the decoded text.

In [18]:
```
%pip install charset-normalizer

from charset_normalizer import from_path

result = from_path("Dataset/students_latin1.txt").best()

if result is not None:
    print("Detected encoding:", result.encoding)
    print("Text:", str(result))
else:
    print("Encoding could not be detected")
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: charset-normalizer in C:\Users\himats\AppData\Roam
ing\Python\Python314\site-packages (3.5.2)
Note: you may need to restart the kernel to use updated packages.
Detected encoding: cp1250
Text: Name: André
City: Montréal
Text: Café
```

In [23]:
```
# Read using original encoding
with open("Dataset/students_latin1.txt", "r", encoding="iso-8859-1") as f:
    text = f.read()

# Save using UTF-8
with open("Dataset/students_converted.txt", "w", encoding="utf-8") as f:
    f.write(text)

print("File converted successfully")
```

```
File converted successfully
```

In [24]:
```
with open("Dataset/students_converted.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

```
Name: André
City: Montréal
Text: Café
```

Part 5: Reading TXT Files Using Pandas

TXT files may contain structured data organized into rows and columns.

Common separators:

- Comma `,`
- Semicolon `;`
- Tab `\t`
- Pipe `|`

Pandas provides `pd.read_csv()` to import structured TXT files.

The `sep` parameter specifies the column separator.

In [25]:
```
with open("Dataset/students.txt", "w", encoding="utf-8") as f:
    f.write("Name;Age;City\n")
    f.write("Ahmad;25;Kuopio\n")
    f.write("Anna;22;Helsinki\n")
    f.write("Maria;24;Joensuu\n")
```

In [26]:
```
import pandas as pd

df = pd.read_csv("Dataset/students.txt", sep=";", encoding="utf-8")

df.head()
```

Out[26]:

	Name	Age	City
0	Ahmad	25	Kuopio
1	Anna	22	Helsinki
2	Maria	24	Joensuu

In [27]:
```
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("Data types:")
print(df.dtypes)
```

```
Shape: (3, 3)
Columns: ['Name', 'Age', 'City']
Data types:
Name      str
Age     int64
City      str
dtype: object
```

In [28]:
```
df.isna().sum()
```

Out[28]:
```
Name    0
Age     0
City    0
dtype: int64
```

Part 6: Handling File Errors and Reusable Functions

File reading may fail because:

- The file does not exist.
- The selected encoding is incorrect.

Python uses `try-except` to handle such errors.

We can also create reusable functions to read multiple text files without repeating code.

In [31]:
```
try:
    with open("students_utf8.txt", "r", encoding="utf-8") as file:
        print(file.read())

except FileNotFoundError:
    print("File not found")

except UnicodeDecodeError:
    print("Incorrect file encoding")
```

```
File not found
```

In [32]:
```
def read_text_file(filepath, encoding="utf-8"):
    try:
        with open(filepath, "r", encoding=encoding) as file:
            return file.read()
    except (FileNotFoundError, UnicodeDecodeError) as error:
        print(error)
        return None

content = read_text_file("Dataset/students_utf8.txt")
print(content)
```

```
Name: Ahmad
City: Kuopio
Text: Hyvää päivää
```

Practical Exercises

Exercise 1: Multilingual TXT File

Create a TXT file containing:

Name: José
City: Málaga

Language: Español
Temperature: 25°C
Finnish: Hyvää päivää

Tasks:

1. Save the file using UTF-8.
2. Read the complete file.
3. Read the file line by line.
4. Try reading it using ASCII.
5. Read it using `errors="replace"`.
6. Save another copy using UTF-16.

Exercise 2: Encoding Conversion

1. Create a file using ISO-8859-1.
2. Try reading it using UTF-8.
3. Read it using the correct encoding.
4. Convert it to UTF-8.
5. Verify the converted text.

Exercise 3: Pandas TXT Import

Create a semicolon-separated TXT file containing student names, ages, and cities.

Tasks:

1. Import the file into Pandas.
2. Display the first five rows.
3. Check the DataFrame shape.
4. Display column names and data types.
5. Count missing values.

Lesson Summary

In this lesson, we learned how to:

- Read TXT files using Python.
- Understand different text encodings.
- Identify and handle encoding errors.
- Detect and convert file encodings.
- Import structured TXT files using Pandas.
- Handle file reading errors.

Key Takeaway: Correct encoding is important for reliable data acquisition and processing. Always verify that text files are read correctly before using them for analysis.